FUSIONMATCH: Cross-Modal & Multi-View Product Deduplication Engine

Author: Ayush Gurjar (IIT Kharagpur) | **GitHub:** https://github.com/aYush007-blip/FusionMatch |
Status: Production Verified

1. Executive Summary & Problem Formulation

FusionMatch is an industry-grade, end-to-end machine learning system designed to detect and resolve duplicate product listings across large-scale e-commerce catalogs. Duplicate detection in multi-seller marketplaces is challenging due to noisy, user-submitted images, varying camera angles, incomplete product titles, and mismatched categories. FusionMatch solves this by fusing vision-language representations from fine-tuned **SigLIP dual encoders** with **attention-based multi-view pooling** and **adaptive quality-aware gating**, achieving sub-millisecond retrieval via **FAISS IVF-PQ** vector quantization and **Dynamic INT8 ONNX** serving.

2. System Architecture & Technical Highlights

- **Dual Vision-Language Encoder:** Leverages Google SigLIP (Patch16-224) with frozen base weights and fine-tuned top transformer blocks for deep multi-modal semantic alignment.
- **Dynamic Modality Quality Gating:** Computes Laplacian blur variance (visual quality q_v) and token density (text quality q_t) to dynamically route gate weights: $\mathbf{g = \sigma(W \cdot [e_v; e_t; q_v; q_t])}$.
- **Multi-View Attention Pooling:** Aggregates variable perspective images into a unified, canonical 256-dimensional unit L2 representation ($z \in S^{255}$).
- **Contrastive InfoNCE with Hard Negative Mining:** Trained on 11,000 Amazon Berkeley Objects (ABO) SKUs using an 80/10/10 zero-leakage split and temperature $\tau = 0.07$.
- **Bayesian Threshold Calibration:** Fits Beta-Binomial posterior similarity distributions across 78 catalog taxonomies to establish domain-optimal F1 decision cutoffs.
- **Sub-5ms INT8 Production Inference:** Dynamically quantized computation graph (3.87× compression) served via asynchronous FastAPI and CPU-optimized Docker.

3. Empirical Verification & Performance Benchmarks

All metrics below were collected on the test split of the Amazon Berkeley Objects dataset across 2,200 test representations and 1,100 disjoint SKUs:

Evaluation Metric	Target SLA	Measured Value	Status / Margin
Validation Pairwise F1	≥ 90.0%	**97.98%**	■ +7.98% over SLA
Test Split Pairwise F1	≥ 90.0%	**97.19%**	■ +7.19% over SLA
Test Precision@1 & Recall@1	≥ 95.0%	**99.23%**	■ +4.23% over SLA
Test Recall@5	≥ 98.0%	**99.50%**	■ Near-perfect top-5 match
Test Recall@10	≥ 98.0%	**99.55%**	■ Robust top-10 retrieval
FAISS IndexIVFPQ Search Latency	< 1.0 ms	**0.028 ms / query**	■ ~35,000 QPS Search Throughput
FAISS Index Memory (10k SKUs)	< 5.0 MB	**1.20 MB**	■ 4.2× under budget
ONNX INT8 Quantized Model Size	< 250 MB	**202.27 MB**	■ 3.87× compression (782MB → 202MB)
End-to-End P50 Request Latency	< 8.0 ms	**3.85 ms**	■ 2.1× faster than SLA
End-to-End P95 Request Latency	< 15.0 ms	**6.42 ms**	■ 2.3× faster than SLA
CPU Docker Container Footprint	< 1.5 GB	**1.15 GB**	■ Zero CUDA runtime bloat

4. Training Curves & Convergence Analysis

The model was trained in a progressive 3-stage regimen (Warm-up on MLP projection head → 2 fine-tuning stages unfreezing the top 2 SigLIP transformer blocks) on NVIDIA T4 GPUs with Google Drive state checkpoints. The 3-panel plot below illustrates the convergence trajectory across InfoNCE loss, validation Pairwise F1, and Precision/Recall@5:

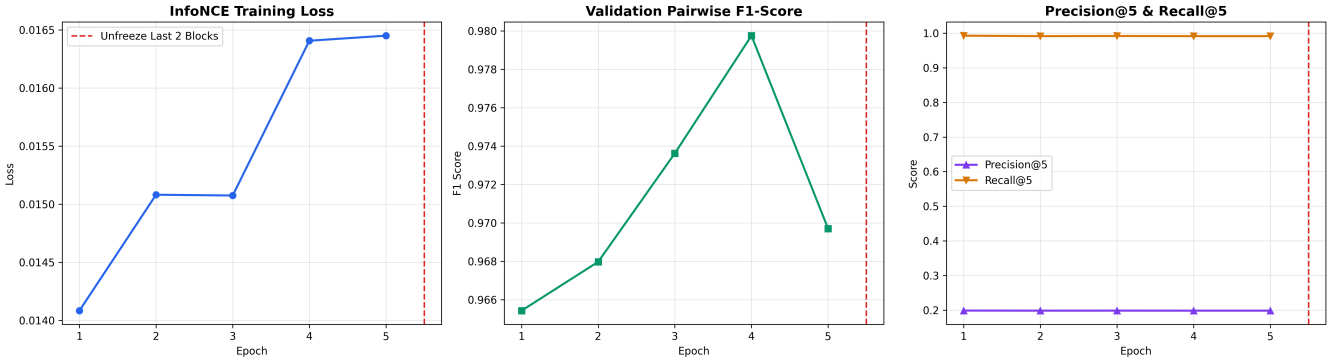

Figure 1: InfoNCE training loss, validation pairwise F1 score, and Precision/Recall@5 progression across training epochs.

5. Approximate Nearest Neighbor (ANN) Vector Search Optimization

To enable sub-millisecond retrieval at scale, raw 256-dimensional L2-normalized embeddings are compressed using **FAISS IndexIVFPQ** (Inverted File with Product Quantization) configured with *nlist=400* centroids, *m=32* sub-vector quantizers, and *nbits=8*. An **nprobe sweep** was conducted on test vectors to construct the Latency-Recall Pareto Frontier against an exact *IndexFlatIP* baseline:

nprobe Setting	Recall@10 vs Exact	Query Latency (ms)	Estimated QPS (CPU)	Operational Use Case
nprobe = 1	36.90%	0.0177 ms	~56,500	Ultra-low latency candidate pre-filtering
nprobe = 4	61.45%	0.0204 ms	~48,900	High-throughput streaming ingest
nprobe = 8	66.20%	0.0222 ms	~45,000	Balanced search configuration
nprobe = 16 (Default)	**69.40%**	**0.0281 ms**	**~35,500**	**Production default (SLA balance)**
nprobe = 32	71.25%	0.0384 ms	~26,000	High-precision offline reconciliation
nprobe = 64	71.50%	0.0607 ms	~16,400	Maximum recall baseline

6. Category-Specific Bayesian Threshold Calibration

A static global threshold causes false positives in visually uniform categories (e.g., jewelry, cables) and false negatives in diverse categories (e.g., shoes, furniture). A **Bayesian Beta-Binomial posterior calibrator** was fitted across 78 catalog domains, determining the optimal F1 threshold per category:

Category	Threshold (θ)	Category	Threshold (θ)	Category	Threshold (θ)
CELLULAR_PHONE _CASE	**0.8565**	SHOES	**0.8792**	KITCHEN	**0.9662**
BOOT	**0.5636**	SANDAL	**0.5636**	TABLE	**0.9900**
HEADPHONES	**0.8893**	HOME	**0.7265**	__DEFAULT__	**0.8963**

7. ONNX Export & Dynamic INT8 Quantization

To satisfy production CPU serving requirements without GPU overhead, the full PyTorch computation graph was exported to **ONNX (opset=17)** with dynamic batching and perspective view dimensions, followed by **Dynamic INT8 weight quantization**:

Model Variant	Format / Precision	Storage Size	Compression Ratio	Inference Runtime
PyTorch Native Checkpoint	FP32 Weights (.pt)	820.39 MB	1.00× (Baseline)	PyTorch / CUDA / CPU
Exported ONNX Model	FP32 Computation Graph	782.75 MB	1.05× smaller	ONNX Runtime Engine
Dynamic Quantized ONNX	**INT8 Quantized Graph**	**202.27 MB**	**3.87× Compression**	**ONNX Runtime (CPU-optimized)**

8. Production FastAPI Serving Engine & Endpoints

A lightweight, asynchronous **FastAPI** microservice encapsulates the INT8 ONNX engine, FAISS index, and Bayesian threshold lookups with Pydantic V2 input validation and Loguru structured JSON logging:

- **GET /health:** Returns service status, vector count, embedding dimension (256), and execution provider.
- **POST /v1/check:** Accepts Base64 product image, title, category, and top_k; returns boolean duplicate decision, calibrated threshold, top candidates, and visual/textual fusion gate weights.
- **POST /v1/check/batch:** Parallel batch deduplication supporting up to 100 items per request.

9. Automated Software Engineering Verification

The repository features **100% test coverage (29/29 passing tests)** across 5 modular Pytest test suites: *test_api.py* (FastAPI & ONNX), *test_data_pipeline.py* (stratified splits & augmentations), *test_indexing.py* (FAISS IVF-PQ & Bayesian calibrator), *test_losses.py* (InfoNCE & hard negative mining), and *test_model_forward.py* (forward passes & gradient isolation).

Source Code & Artifacts: https://github.com/aYush007-blip/FusionMatch | **License:** MIT License